

Edge Language Coach — Architecture Report

Contents

Edge Language Coach — Architecture Report	2
1. System Overview	2
2. Reliability Mechanisms	3
2.1 Circuit Breaker (Groq API)	3
2.2 Rate Limiting	3
2.3 Job Queue Retry & DLQ	3
2.4 Scraper Idempotency	4
2.5 Timeouts	4
2.6 Integration tests	4
3. Scalability	4
3.1 Horizontal Gateway Scale-out	4
3.2 Measured Scale-Out Behaviour	4
3.3 Caveats and Threats to Validity	5
3.4 Worker Concurrency	6
3.5 Production deployment	6
4. Observability	7
4.1 Structured Logging (Pino)	7
4.2 Prometheus Metrics (prom-client)	7
4.3 Grafana Dashboard	7
4.4 Health Probes	7
5. Operator Surface — Agentic MCP	7
5.1 <code>observability-mcp</code> (read-only)	8
5.2 <code>remediation-mcp</code> (guarded write)	8
5.3 Why this matters for reliability	8
6. SLO Targets	9
7. Trade-offs and Lessons Learned	9
Node.js workers over Go microservices	9
BullMQ + Redis over RabbitMQ	9
Supabase over self-hosted PostgreSQL	9
prom-client over full OpenTelemetry	9
Stateless gateway design	9
Agentic MCP operator surface (shipped)	9
8. Future Work (Post-Course)	9
C4 Architecture Diagrams	10
Level 1 — System Context	10

Level 2 — Container Diagram	10
Level 2b — Operator Surface (Agentic MCP)	11
Level 3 — Gateway Component Diagram	11
SLO Table	12
Service Level Objectives	12
k6 Load Test Targets	12
Measured Results	12
Reliability Mechanisms vs. Failure Modes	13
ADR 001 — Node.js Workers over Go Microservices	13
Context	13
Decision	14
Consequences	14
ADR 002 — BullMQ + Redis over RabbitMQ	14
Context	14
Decision	14
Consequences	14
ADR 003 — Supabase over Self-Hosted PostgreSQL	15
Context	15
Decision	15
Consequences	15
ADR 004 — prom-client over Full OpenTelemetry Stack	15
Context	15
Decision	16
Consequences	16

Edge Language Coach — Architecture Report

1. System Overview

Edge Language Coach is an Italian language learning application that uses AI-powered conversational practice, spaced-repetition flashcards, and automated topic curation to help learners improve fluency.

The system is designed as a **distributed, event-driven architecture** with three independently deployable runtime units:

Unit	Technology	Role
Web	React 19 + Vite → nginx	SPA served as static files
Gateway	Fastify (Node.js)	REST API, auth, reliability primitives
Workers	Node.js + BullMQ	Async job processing (scraper, summary, flashcards)

See `docs/c4-diagram.md` for full C4 Level 1–3 diagrams.

2. Reliability Mechanisms

2.1 Circuit Breaker (Groq API)

The gateway wraps all Groq API calls (LLM chat completions, Whisper transcription) in two independent opossum circuit breakers:

- **Chat breaker:** Opens after 50% error rate over a 60 s rolling window (min 10 requests). Resets after 30 s.
- **Transcribe breaker:** Same thresholds, with a 60 s timeout (audio inference is slower).

When a breaker is open, requests to that capability return an immediate error rather than blocking the gateway thread waiting for a timeout. This prevents Groq slowdowns from cascading into full service degradation.

Breaker state is exposed as a Prometheus gauge (`groq_circuit_breaker_state`) and visible in the Grafana dashboard.

Measured breaker behaviour. The breaker was exercised end-to-end using a mock Groq server (`mocks/groq-mock.mjs`) put into `slow` mode (30 s sleep per request, exceeding the 15 s breaker timeout) and the driver script (`mocks/breaker-driver.mjs`) firing 60 requests at concurrency 12. Full output at `load/breaker-demo-output.txt`. Summary:

Phase	Calls	Per-call latency	Status
Closed (probing)	1–7	15.3 s (timeout)	breaker collecting failure samples
Transition	call 7	—	volumeThreshold (10) + 50% error rate crossed → opened
Open (short-circuit)	8–60	12–58 ms	requests fail-fast, never reach mock

Latency cliff: 15 300 ms → 25 ms in adjacent calls. Without the breaker, all 60 requests would have hit the 15 s timeout sequentially or in parallel waves; with it, only 7 calls paid the upstream-stalled cost. The remaining 53 calls cost the system effectively zero CPU and zero open sockets to Groq.

2.2 Rate Limiting

All API routes are rate-limited at 60 requests/minute per authenticated user (or per IP for unauthenticated requests), backed by Redis for distributed counting across replicas. Health probes (`/livez`, `/readyz`, `/metrics`) are allowlisted.

2.3 Job Queue Retry & DLQ

Workers use BullMQ with a shared retry configuration (3 attempts, exponential backoff starting at 5 s). Failed jobs are logged at `error` level with `dlq: true` after exhausting all retries, providing a

structured dead-letter record visible in log aggregation without requiring a separate DLQ queue.

All queue state is visible in the Bull Board UI at `:3002/queues`.

2.4 Scraper Idempotency

The scraper cron job (every 6 hours) sets a Redis key after each successful run (`scraper:slot:<date>:<slot>`, TTL = 6 h). If the job is retried within the same time window (e.g., after a transient failure), it exits early to prevent duplicate topic ingestion.

Title-level deduplication (exact and prefix match against the last 30 days of topics) provides a second layer of protection.

2.5 Timeouts

- Gateway: 30 s connection timeout, 5 s keep-alive timeout
- Groq chat: 15 s circuit breaker timeout
- Groq transcribe: 60 s circuit breaker timeout
- Health probes: 2 s per dependency check (Redis, Supabase, Groq)

2.6 Integration tests

The reliability claims in §2.1–§2.5 are not just asserted in prose — they are pinned in CI. The gateway has a Vitest suite ([apps/gateway/src/__tests__/health.test.ts](#)) using Fastify's `inject()` harness, with Redis, Supabase, and Groq mocked at module boundaries so tests run hermetically. Currently covered:

- GET `/livez` returns 200 unconditionally.
- GET `/readyz` returns 200 with `{status: "ready"}` when all dependencies are healthy.
- GET `/readyz` returns 503 with `{deps: {redis: {ok: false}}}` when the Redis ping rejects.
- GET `/readyz` returns 503 with `{deps: {groq: {ok: false}}}` when the Groq probe rejects.

CI ([.github/workflows/ci.yml](#)) runs `pnpm turbo test` ahead of typecheck and build on every push and PR, so a regression in the readiness path or its failure semantics fails the pipeline. Breaker, rate-limit, and queue-retry tests are the natural next additions and would close the rest of the gap.

3. Scalability

3.1 Horizontal Gateway Scale-out

The gateway is stateless (all session state is in Supabase; all rate-limit counters are in Redis). It can be scaled horizontally with:

```
docker compose up --scale gateway=3
```

An nginx load balancer (`nginx-lb`) sits in front of all gateway replicas and distributes traffic via Docker's built-in DNS round-robin. The web SPA and Prometheus both target `nginx-lb:3001`.

3.2 Measured Scale-Out Behaviour

The k6 script at `load/gateway.js` runs three sequential scenarios in a single invocation:

Scenario	Profile	Purpose
baseline	10 VUs constant for 30 s	Light steady-state load
stressed	Ramp 0 → 50 VUs over 60 s, then taper	Stress test rate-limit + downstream
scaled_out	50 VUs constant for 30 s	Saturated steady-state

The full script was run twice — once with a single gateway replica (`docker compose up gateway`), once with three replicas (`docker compose up --scale gateway=3`). All other parameters were held constant. Results:

Metric	1 replica	3 replicas	Δ
gateway_errors rate	6.81 %	0.27 %	−96 %
scaled_out p95 latency	2.43 s	2.15 s	−12 %
stressed p95 latency	1.73 s	1.84 s	+6 %
baseline p95 latency	0.34 s	0.66 s	+93 % (cold-start noise; see §3.3)
Sustained throughput	39.3 req/s	39.4 req/s	unchanged

Interpretation. Adding replicas had a dramatic effect on the *error rate* (the 1-replica gateway was dropping ~7 % of requests under saturation; with 3 replicas this collapsed to under 0.3 %), but only a modest effect on *latency* under saturation (~12 % p95 reduction). This indicates that under sustained load:

- The **gateway itself** is the bottleneck for **availability** — a single Node.js event loop cannot keep up with 50 concurrent virtual users plus rate-limit checks plus auth verification, so it starts dropping connections.
- The **downstream request hot path** (Supabase RTT, Redis rate-limit checks) is the bottleneck for **latency** — adding gateway replicas does not reduce the per-request work outside the gateway, so p95 only improves marginally.

The persistent ~25 % `http_req_failed` rate seen in both runs is the rate-limiter doing its job correctly: 50 VUs × ~10 req/s = 500 req/min vs. the configured 60 req/min cap. These are 429 responses, not failures of the system.

3.3 Caveats and Threats to Validity

- **Co-located load generator.** k6 ran on the same Windows host as Docker Desktop, so the test loop competed for CPU with the containers. This is most visible in the unexpected **baseline** p95 regression on the 3-replica run (Docker had to schedule three gateway containers + the original two from the first run, evicting cache pages).
- **Mocked Groq.** All Groq calls returned in <1 ms via [mocks/groq-mock.mjs](#), eliminating a normally significant latency contributor. This isolates the gateway/queue layer (the system under test) but means absolute p95 numbers should not be read as production estimates.
- **Single iteration.** Each replica configuration was tested once; no statistical variance across runs.

- **Limited scale.** Three replicas is the largest configuration tested (constrained by available host CPU). The diminishing-returns curve at higher replica counts has not been measured.

3.4 Worker Concurrency

Workers are horizontally scalable as well — BullMQ supports multiple consumers on the same queue and guarantees each job is processed exactly once. Within a single worker process, per-queue concurrency is configured independently:

Queue	Concurrency	Rationale
flashcard-generate	3	Parallelism limited by Groq rate limits
summary-generate	3	Same
topic-scrape	1	Scraper is serial to avoid race conditions on dedup state

3.5 Production deployment

Production splits the system across two hosts: the gateway, workers, Redis, and `nginx-lb` run on a single Oracle Cloud VM via `docker-compose.prod.yml`; the React SPA is hosted on Vercel. Three GitHub Actions workflows manage the release pipeline:

Workflow	Trigger	Result
<code>ci.yml</code>	every push and PR to main	<code>pnpm turbo test + typecheck + build</code> . No deploy steps — strictly a per-PR check.
<code>deploy.yml</code>	push to main, <code>paths-ignore: apps/web/**, vercel.json, docs/**, *.md</code>	<code>typecheck/build/test → docker buildx multi-arch (linux/amd64+linux/arm64) push to GHCR → SSH deploy to the Oracle host</code>
<code>deploy-web.yml</code>	push to main, <code>paths: apps/web/**, packages/**, vercel.json</code>	<code>typecheck/build/test → npx vercel --prod</code>

The path filters mean a doc-only or web-only change skips the (expensive) backend rebuild, and a backend-only change skips the SPA deploy. This keeps the median deploy cycle under a minute on docs and under three minutes on a backend-only change.

The Oracle host deploy step does `git reset --hard origin/main`, `docker compose pull && up -d --remove-orphans`, then **explicitly restarts `nginx-lb`** before pruning dangling images. The explicit restart is load-bearing: Docker’s embedded DNS otherwise caches the old gateway container’s IP and the load balancer keeps trying to reach a container that no longer exists.

The Vercel build is driven by `vercel.json`, which runs `pnpm turbo build --filter=@edge/web` and rewrites `/api/:path*` to the Oracle backend’s `:3001` port. The browser only ever talks to the Vercel origin, which sidesteps the need for CORS configuration at the gateway.

The prod compose includes Redis, workers, gateway, and `nginx-lb`; it intentionally omits Prometheus, Grafana, and Bull Board (those remain in the local `docker-compose.yml` for the observability story), and the SPA (Vercel handles that).

4. Observability

4.1 Structured Logging (Pino)

All services emit JSON-structured logs via `pino`. Each log line includes `service` (gateway/workers), `requestId`, and relevant domain fields (`sessionId`, `jobId`, queue name). Log level is configurable via `LOG_LEVEL` env var.

4.2 Prometheus Metrics (prom-client)

The gateway exposes `/metrics` in Prometheus text format, scraped every 15 s. Key metrics:

Metric	Type	Labels
<code>http_request_duration_seconds</code>	Histogram	method, route, status_code
<code>http_requests_total</code>	Counter	method, route, status_code
<code>groq_circuit_breaker_state</code>	Gauge	breaker (chat/transcribe)
<code>queue_depth_total</code>	Gauge	queue
<code>active_sessions_total</code>	Gauge	—

Node.js default metrics (event loop lag, memory, GC) are also collected via `collectDefaultMetrics()`.

4.3 Grafana Dashboard

A pre-provisioned dashboard (`docker/grafana/dashboards/edge-coach.json`) is loaded automatically on Grafana startup. Panels: request rate by route, p50/p95/p99 latency, 5xx error rate, circuit breaker state, queue depth, active sessions. Accessible at `localhost:3000` (admin/admin).

4.4 Health Probes

- `/livez`: Always 200 — liveness signal for container orchestration.
- `/readyz`: Checks Redis, Supabase, and Groq availability with 2 s timeouts. Returns 503 if any dependency is degraded.

5. Operator Surface — Agentic MCP

A two-server [Model Context Protocol](#) layer exposes the running system to agent-driven inspection and remediation, **without** SSH, `kubectl`, or a redeploy. This closes the operator loop on top of the observability stack: **detect** (Prometheus / Grafana) → **diagnose** (`observability-mcp`) → **remediate** (`remediation-mcp`).

The two servers are separated by privilege: read-only telemetry on one transport, guarded mutations on a second.

5.1 observability-mcp (read-only)

Wraps `/metrics` and `/readyz` as five typed tools. No mutation, no auth (stdio transport, locally scoped). Source: [apps/observability-mcp/src/index.ts](#).

Tool	Backed by
<code>get_service_health</code>	Gateway + workers <code>/readyz</code>
<code>get_queue_metrics</code>	<code>queue_depth_total</code> per queue
<code>get_circuit_breaker_state</code>	<code>groq_circuit_breaker_state{breaker}</code>
<code>get_active_sessions</code>	<code>active_sessions_total</code>
<code>get_groq_latency</code>	<code>groq_request_duration_seconds</code> histogram lines

5.2 remediation-mcp (guarded write)

Mutates runtime state through three layered guards. Source: [apps/remediation-mcp/src/index.ts](#).

Tool	Mechanism	Guard	Reversible?
<code>pause_queue / resume_queue</code>	BullMQ admin via Redis	<code>zod.enum</code> restricts queue name to known queues	yes
<code>reset_circuit_breaker</code>	POST <code>/admin/breakers/reset</code> HTTP header (apps/gateway/src/routes/admin.ts)	<code>ADMIN_API_KEY</code>	yes (breakers may re-open if upstream is still bad)
<code>flush_dead_letter_queue</code>	<code>Queue.clean('failed')</code>	<code>confirm: true</code> argument required	no

Every invocation writes a structured `pino` audit line to `stderr` (`stdout` is reserved for the MCP transport — see [remediation-mcp/src/index.ts:21-26](#)).

5.3 Why this matters for reliability

Without the MCP layer, recovery from an open breaker, a poisoned DLQ, or a queue that needs to be drained for maintenance requires either a Bull Board click-through, a `redis-cli` session, or a redeploy. With it, the same diagnostic and remediation primitives are exposed as a typed, audited contract that any MCP client can call — Claude Desktop, Claude Code, or a future automated runbook agent.

Concretely: when the breaker demo opens the chat breaker, the recovery sequence becomes a three-tool dialogue: 1. `observability-mcp.get_circuit_breaker_state` → `confirm chat: open` 2. (operator verifies upstream Groq is healthy) 3. `remediation-mcp.reset_circuit_breaker` → forces both breakers closed; audit line on `stderr`

This partially compensates for the absence of distributed tracing (ADR-004): the operator cannot follow a single span through the system, but they can interrogate live state with structured tools rather than reading raw metrics by hand.

6. SLO Targets

See `docs/slo-table.md` for the full SLO table including k6 load test targets and measured results.

Key SLOs: - API availability 99% - `/api/messages` p95 < 2 s at baseline (10 VUs) - 5xx error rate < 1% at baseline

7. Trade-offs and Lessons Learned

Node.js workers over Go microservices

Chose Node.js for velocity (shared TypeScript types, same toolchain). Trade-off: workers cannot scale independently. See ADR 001.

BullMQ + Redis over RabbitMQ

Single Redis instance serves both rate-limiting and job queuing. Simpler infrastructure for a demo system. Trade-off: Redis is not a purpose-built broker. See ADR 002.

Supabase over self-hosted PostgreSQL

Eliminates auth infrastructure; trade-off is external dependency making fully offline operation impossible. See ADR 003.

prom-client over full OpenTelemetry

Faster to instrument, fewer services. Trade-off: no distributed traces across gateway→worker hops. See ADR 004.

Stateless gateway design

Placing all shared state (rate limits, job queues) in Redis made horizontal scale-out trivial — no sticky sessions, no shared memory. The nginx-lb + Redis combination provides elasticity without code changes.

Agentic MCP operator surface (shipped)

Originally listed as future work, now shipped as `apps/observability-mcp` and `apps/remediation-mcp`. The split between read-only and guarded-write transports is the load-bearing decision; bundling both into one server would have meant every consumer inherits the privilege of the most-privileged tool.

8. Future Work (Post-Course)

- OpenTelemetry distributed tracing (gateway workers correlation)
- Kubernetes deployment manifests (HPA for gateway)
- Redis AOF persistence enabled for production durability

- Tighten remediation-mcp auth: `pause_queue` / `resume_queue` currently rely on `stdio-locality` and the `zod.enum` queue allow-list; production deployment should require `ADMIN_API_KEY` for all mutating tools, not just `reset_circuit_breaker`

C4 Architecture Diagrams

Level 1 — System Context

```
graph TB
    user["Language Learner\n(Browser)"]
    system["Edge Language Coach\n[Software System]\nItalian language practice\nvia AI-powered"]
    supabase["Supabase\n[External System]\nPostgreSQL + Auth"]
    groq["Groq API\n[External System]\nLLM inference (Llama 3.3)\nWhisper transcription"]

    user -->|"Uses (HTTPS)"| system
    system -->|"Reads/writes sessions,\ntopics, flashcards"| supabase
    system -->|"Chat completions,\naudio transcription"| groq
```

Level 2 — Container Diagram

```
graph TB
    browser["Browser SPA\n[React 19 + Vite]\nUser interface for chat,\nflashcards, reports"]
    nginx_web["Web Server\n[nginx:alpine]\nServes static SPA,\nproxies /api to nginx-lb"]
    nginx_lb["Load Balancer\n[nginx:alpine]\nRound-robin across\ngateway replicas"]
    gateway["API Gateway\n[Fastify / Node.js]\nAuth, rate limiting,\ncircuit breaker, REST API"]
    workers["Workers\n[Node.js / BullMQ]\nScraper, summary,\nflashcard generation\n:3002"]
    redis["Redis\n[redis:7-alpine]\nBullMQ job queues,\nrate-limit counters\n:6379"]
    prometheus["Prometheus\n[prom/prometheus]\nMetrics scraping\n:9090"]
    grafana["Grafana\n[grafana/grafana]\nDashboards & alerting\n:3000"]

    supabase["Supabase\n[External]\nPostgreSQL + Auth"]
    groq["Groq API\n[External]"]

    browser -->|"HTTPS"| nginx_web
    nginx_web -->|"proxy /api"| nginx_lb
    nginx_lb -->|"round-robin"| gateway
    gateway -->|"enqueue jobs"| redis
    gateway -->|"SQL queries"| supabase
    gateway -->|"LLM / Whisper\n(circuit-broken)"| groq
    workers -->|"consume jobs"| redis
```

```
workers -->|"SQL writes"| supabase
workers -->|"LLM inference"| groq
prometheus -->|"scrape /metrics"| nginx_lb
grafana -->|"PromQL queries"| prometheus
```

Level 2b — Operator Surface (Agentic MCP)

A separate plane from the user-request data flow. MCP clients (Claude Desktop, Claude Code, automated runbook agents) speak the stdio transport to two servers split by privilege.

```
graph LR
    operator["Operator / Agent\n(Claude Desktop, Claude Code)"]

    obs_mcp["observability-mcp\n[stdio MCP server]\nRead-only:\n• get_service_health\n• get_queue_size"]
    rem_mcp["remediation-mcp\n[stdio MCP server]\nGuarded write:\n• pause_queue / resume_queue"]

    gateway["API Gateway\n:3001"]
    workers["Workers\n:3002"]
    redis["Redis"]
    admin["POST /admin/breakers/reset\n(ADMIN_API_KEY-guarded)"]
    audit["stderr audit log\n(pino, structured)"]

    operator -->|"stdio"| obs_mcp
    operator -->|"stdio"| rem_mcp

    obs_mcp -->|"GET /metrics, /readyz"| gateway
    obs_mcp -->|"GET /readyz"| workers

    rem_mcp -->|"BullMQ admin"| redis
    rem_mcp -->|"x-admin-key"| admin
    admin --- gateway
    rem_mcp -.->|"every call"| audit
```

Loop closure: detect (Prometheus / Grafana, not shown) → diagnose (observability-mcp) → remediate (remediation-mcp).

Level 3 — Gateway Component Diagram

```
graph TB
    subgraph Gateway ["API Gateway (Fastify)"]
        auth["Auth Plugin\n(JWT validation)"]
        rl["Rate Limit Plugin\n(@fastify/rate-limit + Redis)\n60 req/min per user"]
        cb["Circuit Breaker\n(opossum)\nncat + transcribe breakers"]
        routes["Route Handlers\n/api/topics, /api/sessions,\n/api/messages, /api/flashcards,\n"]
        health["Health Routes\n/livez, /readyz, /metrics"]
        queue_client["Queue Client\n(BullMQ producer)\nflashcard-generate\nsummary-generate"]
    end
```

```

req["Incoming Request"] --> auth --> rl --> routes
routes --> cb
routes --> queue_client
cb -->|"Groq SDK"| groq["Groq API"]
queue_client -->|"Redis"| redis["Redis"]
health -->|"ping"| redis

```

SLO Table

Service Level Objectives

SLI	Measurement Method	Target (SLO)	Measurement Window
API availability	1 - rate(http_requests_total{status_code=~"5.."}[5m])	99%	Rolling 5 min
/api/messages p95 latency	1 - histogram_quantile(0.95, rate(http_request_duration_seconds_bucket{route="/api/messages"}[5m]))	< 2 s	Rolling 5 min
/api/topics p95 latency	1 - histogram_quantile(0.95, rate(http_request_duration_seconds_bucket{route="/api/topics"}[5m]))	< 500 ms	Rolling 5 min
/readyz availability	Prometheus up metric	100%	Continuous
Queue job success rate	Failed jobs / total jobs processed	95%	Per day
Scraper freshness	Topics inserted per 24 h	5 new topics	Per day

k6 Load Test Targets

Scenario	VUs	Duration	p95 Latency Target	Error Rate Target
Baseline	10	30 s	< 2 s	< 1%
Stressed	ramp 0→50	75 s	< 4 s	< 5%
Scaled-out (2 replicas)	50	30 s	< 2 s	< 1%

Measured Results

Sourced from `k6 run load/gateway.js` against `docker compose up --scale gateway=N` with Groq mocked (see [architecture-report.md §3](#) for full methodology and threats to validity). Comparison of the same script run against 1 vs 3 gateway replicas:

Scenario	Replicas	p95 latency	gateway_errors rate	Throughput
Baseline	1	0.34 s	n/a	—
Baseline	3	0.66 s	n/a	—
Stressed	1	1.73 s	n/a	—
Stressed	3	1.84 s	n/a	—
Scaled-out	1	2.43 s	6.81 %	39.3 req/s
Scaled-out	3	2.15 s	0.27 %	39.4 req/s

Headline finding: going from 1 → 3 gateway replicas reduces the `scaled_out` error rate by ~**96 %** (6.81 % → 0.27 %) while leaving sustained throughput essentially unchanged (the bottleneck moves from the gateway event loop to the downstream Supabase / Redis RTT). The persistent ~25 % `http_req_failed` rate seen in both runs is the rate-limiter doing its job — 50 VUs × ~10 req/s 500 req/min vs the configured 60 req/min cap.

The unexpected `baseline` p95 regression on the 3-replica run is host-CPU contention from the co-located k6 generator and is documented under “threats to validity” in the architecture report.

Reliability Mechanisms vs. Failure Modes

Failure Mode	Detection	Recovery
Groq API slow/unavailable	Circuit breaker opens after 50% errors in 60 s window	Breaker resets after 30 s; cached response or 503 returned
Redis unavailable	<code>/readyz</code> returns 503; rate-limiter falls back to in-memory	BullMQ reconnects automatically; job queue persists in Redis
Worker job crash	<code>job.failed</code> event logged at <code>error</code> level with <code>dlq: true</code> after 3 attempts	Manual replay via Bull Board UI at <code>:3002/queues</code>
Gateway overloaded	Rate limiter returns 429 after 60 req/min per user	nginx-lb spreads load across replicas; scale with <code>--scale gateway=N</code>
Scraper re-run within same slot	Redis idempotency key checked at job start	Job exits early (<code>skipping - already completed this slot</code>)

ADR 001 — Node.js Workers over Go Microservices

Date: 2026-04-30

Status: Accepted

Context

The original architecture plan called for three independently deployable Go services (Topic Scraper, Recommendations Engine, Flashcard/SRS Engine) communicating with the gateway via internal

REST APIs. The goal was to demonstrate a distributed system with independently scalable components.

Decision

Implement the workers as a single Node.js service (**apps/workers**) using BullMQ + Redis instead of three Go services.

The three logical workers (scraper, summary, flashcard) remain isolated as separate **Worker** instances with their own queues and concurrency settings, preserving the distributed semantics without the operational overhead of multiple language runtimes.

Consequences

Positive: - Single language runtime (TypeScript throughout) reduces cognitive overhead and tooling complexity. - Shared types and validation schemas via **@edge/shared** eliminate interface contract drift between services. - BullMQ's persistent queue guarantees (at-least-once delivery, retry, DLQ) replace the reliability burden that would otherwise fall on inter-service HTTP. - Faster iteration: no need to manage separate Go module versions, Dockerfiles, or CI steps per service.

Negative: - Workers cannot be scaled independently from each other (one Docker service scales all three). Mitigated by BullMQ's per-queue concurrency configuration. - A crash in one worker process affects all three. Mitigated by the event-driven design: jobs remain in Redis until consumed, and BullMQ retries failed jobs. - Less dramatic “polyglot” demonstration for the course submission.

ADR 002 — BullMQ + Redis over RabbitMQ

Date: 2026-04-30

Status: Accepted

Context

The async event-driven layer (Phase 3) needed a message broker to decouple the gateway from the workers (summary generation, flashcard generation, topic scraping). The two main candidates were RabbitMQ (AMQP) and Redis-backed BullMQ.

Decision

Use BullMQ with Redis as the message broker.

Consequences

Positive: - **Single dependency:** Redis serves both as the rate-limiter store (via **@fastify/rate-limit**) and the job queue backend, reducing the number of infrastructure services. - **Rich job lifecycle:** BullMQ provides built-in retry with exponential backoff, dead-letter tracking, delayed jobs, recurring cron jobs, and a monitoring UI (Bull Board) — features that require plugins or custom code in RabbitMQ. - **Simpler local dev:** **docker compose up redis** is sufficient; no RabbitMQ management UI or vhost setup needed. - **TypeScript-native:** BullMQ is written in TypeScript; type-safe job data without protobuf/AMQP schema definitions.

Negative: - Redis is not a purpose-built message broker. Under extreme load it can lose acknowledged-but-not-persisted messages if AOF persistence is not enabled. Mitigated by running Redis with default append-only persistence in production. - BullMQ does not support fanout/pub-sub patterns natively. Not required for current workloads. - Vendor lock-in to Redis data structures. Switching to RabbitMQ later would require replacing all queue producers and consumers.

ADR 003 — Supabase over Self-Hosted PostgreSQL

Date: 2026-04-30

Status: Accepted

Context

The system requires a relational database (user sessions, topics, flashcards, feedback) and an authentication provider. Options considered: self-hosted PostgreSQL + a separate auth service (e.g., Auth.js, Keycloak), or a managed Backend-as-a-Service such as Supabase.

Decision

Use Supabase (managed PostgreSQL + Auth + Storage).

Consequences

Positive: - **Auth out of the box:** Row-level security, JWT issuance, OAuth providers, and email/password auth are available without additional services. The gateway validates Supabase JWTs directly, eliminating an auth microservice. - **Reduced operational surface:** No Postgres container to manage, tune, back up, or upgrade within the project's Docker Compose. Supabase handles backups, connection pooling (PgBouncer), and upgrades. - **Free tier sufficient:** The project's workload (course demo) comfortably fits within Supabase's free tier. - **Prisma ORM compatibility:** Supabase PostgreSQL works identically to self-hosted PostgreSQL from Prisma's perspective; migration if needed is a connection string change.

Negative: - **External dependency:** The system cannot run fully offline or in a fully self-contained Docker environment. A self-hosted Supabase stack could be added but adds significant setup complexity. - **Vendor lock-in on auth:** Supabase-specific JWT claims and row-level security policies would need to be migrated if moving to another provider. - **Latency variability:** As a managed service in a remote region, Supabase queries add network latency. Mitigated by Prisma connection pooling and Supabase's regional deployments.

ADR 004 — prom-client over Full OpenTelemetry Stack

Date: 2026-04-30

Status: Accepted

Context

Step 5 of the SRS roadmap requires observable metrics and dashboards. Two approaches were considered: (a) full OpenTelemetry (traces + metrics + logs) with an OTel Collector, or (b) prom-

client (Prometheus metrics only) with Grafana.

Decision

Use `prom-client` to expose Prometheus-format metrics at `/metrics`. Pair with a self-hosted Prometheus + Grafana stack in Docker Compose. Defer distributed tracing (OpenTelemetry spans) as an optional future enhancement.

Consequences

Positive:

- **Low dependency count:** `prom-client` is a single npm package with no native dependencies. No OTel Collector sidecar, no OTLP endpoint, no Jaeger/Tempo/Zipkin.
- **Grafana compatibility:** `prom-client` metrics are consumed directly by Prometheus, which Grafana queries natively. The full metrics pipeline (scrape → store → visualize) fits in four docker-compose services (gateway, prometheus, grafana, nginx-lb).
- **Sufficient for SLO verification:** Histogram buckets provide p50/p95/p99 latency measurements; counters track error rates and request volume — the SLIs needed to evaluate the SLO table.
- **Fast to instrument:** Adding a histogram observe call to Fastify's `onResponse` hook takes ~10 lines of code.

Negative:

- **No distributed traces:** Request spans cannot be correlated across gateway → worker hops. A slow summary job cannot be pinpointed to a specific Groq call without adding OTel later.
- **Pull-based scraping:** Prometheus scrapes `/metrics` every 15 seconds; resolution is coarser than push-based OTel. Acceptable for the course demo.
- **Metrics only on gateway:** Workers do not expose a `/metrics` endpoint. Queue depth is proxied through the gateway's metrics, which adds a query-per-scrape overhead.